Project Description

A managed collection of PhotoImages for use with Tkinter widgets.

This package furnishes a convenient way of providing PhotoImages to Tkinter widgets that display images, such as Buttons, Labels, and Menus.

The **ImageList** class incorporates the capabilities of the **Python Imaging Library (PIL)** for both loading images from a wide variety of commonly used image file formats, and for resizing and reformating the original images into Tkinter compatible PhotoImages. Regardless of the original source image dimensions, every image that is added to the collection is resized (if needed) to the **image_size** that was specified when the **ImageList** was created. In addition to providing PhotoImages of uniform size, the **ImageList** class also provides image object persistence for Tkinter widgets by maintaining a reference for each PhotoImage in the collection.

Attention macOS Users : When working with a Tkinter widget on either a **Windows** or a **Linux** based platform, the widget will display a "grayed" image when the widget's **state** option is set to **'disabled'**. This behavior visually indicates whether the widget is active or inactive. However, when using that same Tkinter widget on a **macOS** computer, the image displayed by the widget is left unchanged when the widget's **state** option is set to **'disabled'**. The **ImageList** class was designed to help resolve this issue by maintaining a corresponding collection of "grayed" PhotoImages that are also created from the image files. By assigning the corresponding "grayed" PhotoImage to the **image** option of a **'disabled'** widget, the widget can visually indicate that it is not active.

Installation

```
pip install imagelist
```

Overview

This package provides the following class definition **:**

- **ImageList -** A managed collection of PhotoImages for use by Tkinter widgets

Note : A *top-level* or *root* window must be in existence prior to creating an instance of the **ImageList** class.

The **ImageList** class provides a programming interface that is similar to that of a Python **list** object. The PhotoImage elements in an **ImageList** collection are ordered and indexed, and a specific PhotoImage can be accessed by referring to its index number. Both positive and negative index values are supported. The **ImageList** collection will issue a warning message and return a blank PhotoImage when referenced with an invalid index value. The **len( )** function can is used to determine the total number of PhotoImage elements in the **ImageList** collection.

For example:

```
# Create a collection of PhotoImages that are all 16 x 16 pixels in size

image_list = ImageList()  # The default image size is 16 x 16 pixels
image_list.add('image_file_0')  # Add the first image from image_file_0
image_list.add('image_file_1')  # Add the next  image from image_file_1
# Continue to add more images to the collection
...

image_count = len(image_list)  # Determine the total number of PhotoImages

first_image = image_list[0]  # Get the first PhotoImage in the collection
second_image = image_list[1]  # Get the second PhotoImage in the collection

last_image = image_list[-1]  # Get the last PhotoImage in the collection
```

A specific PhotoImage element in an **ImageList** collection can also be accessed by referencing its (optional) key name. An image's key name can be specified when the image is added to the collection, or it can be assigned at a later time by using the **set_key_name(index, name)** method. Unlike a dictionary, the **ImageList** collection does not require every element to have an unique key name value, and the key name values are not case sensitive. When an image is added without a specified key name, the collection will assign an empty string as that image's key name value. When accessing a PhotoImage in the collection by a key name value, the first PhotoImage with a matching key name value is returned. The **ImageList** collection will issue a warning message and return a blank PhotoImage when referenced with either an empty string or a non-matching key name value. The **keys** property returns a list of the key names that are currently assigned to the PhotoImages in the collection.

The following example shows the use of key name values:

```
image_list = ImageList()
image_list.add('image_file_0', 'key_name_0')  # Specify image_0's key name
image_list.add('image_file_1', 'key_name_1')  # Specify image_1's key name
# Continue to add more images to the collection
...

first_image = image_list['key_name_0']  # Get the first PhotoImage
second_image = image_list['key_name_1']  # Get the second PhotoImage

image_list.set_key_name(0, 'new_name')  # Change the first PhotoImage's key name
```

The **ImageList** class's **grayed** property provides access to the "grayed" PhotoImage collection. Each PhotoImage element in the **ImageList** collection has a corresponding "grayed" PhotoImage in the **ImageList.Grayed** collection. These paired images share the same index number and key name values.

This example illustrates how the **ImageList** can to used to provide a "grayed" image to a Tkinter Button widget when it is made inactive:

```
import tkinter as tk
import platform

image_list = ImageList()
image_list.add('image_file', 'key_name')  # Add an image file and a key name

button = tk.Button(parent, image=image_list['key_name'], command=command)

# Make the button inactive and display a "grayed" image
button.configure(state='disabled')
if platform.system() == 'Darwin':  # Check for the macOS platform
    button.configure(image=image_list.grayed['key_name'])
```

API Documentation

ImageList

ImageList(resource_folder=' ', image_size=(16, 16), auto_load=False)

Constructs and initializes the PhotoImage collection.

- **resource_folder: str -** The path of the resource file folder for the image files, **default = ' '.**
- **image_size: tuple[int, int] -** The size (width, height) of the PhotoImages, **default = (16, 16) pixels.**
- **auto_load: bool -** Automatically load all the resource folder's image files if True, **default = False.**

The **resource_folder** string is used internally by the **add(image_file, key_name=None)** method to construct the full path name when adding an image to the collection.

```
full_path_name = os.path.join(resource_folder, image_file)
```

When the **auto_load** parameter's value is set **True**, all the valid image files in the specified **resource_folder** will be identified and automatically added to the collection. The image files are added to the collection in alphabetical order. The **key_name** value assigned to each added image will be the image file's *stem* or *base* name (the image filename without its extension).

```
extension = filename.rfind('.')
key_name = filename[:extension]
```

Properties

- **blank_image: PhotoImage -** A blank (or transparent) PhotoImage. (readonly)
- **grayed: ImageList.Grayed -** A managed collection of grayed PhotoImages. (readonly)
- **image_size: tuple[int, int] -** The size (width, height) of the PhotoImages in the collection. (readonly)
- **keys: list[str] -** A list of the key names currently assigned to the PhotoImages. (readonly)
- **resource_folder: str -** The path of the resource file folder for the image files. (read / write)

Methods

- **add(image_file, key_name=None) -> bool :** Add an image with an optional key name to the end of the collection. The full path name of the image file is constructed from the **resource_folder** property value and the **image_file** parameter value.

    - **image_file: str -** The image file to add to the collection.
    - **key_name: str | None -** The optional key name of the image (not case-sensitive).

    Returns : True if the image was successfully added **, False** otherwise

- **clear( ) :** Remove all the images and key names from the collection.

- **contains_key(name) -> bool :** Determine if the collection has an image with the specified key name.

    - **name: str -** The specified key name of the image (not case-sensitive).

    Returns : True if the collection contains the key name **, False** otherwise.

- **extend(image_list) -> bool :** Add a PhotoImage collection to the end of the current collection. The **image_size** property of the PhotoImage collection must match that of the current collection in order to be successfully added.

    - **image_list: ImageList -** The specified PhotoImage list.

    Returns : True if the PhotoImage collection was successfully added **, False** otherwise.

- **index_of_key(name: str) -> int :** Return the zero-based index of the image with the specified key name.

    - **name: str -** The specified key name of the image (not case-sensitive).

    Returns : The **index** of the first occurrence of the key name **, -1** otherwise.

- **remove_at(index: int) :** Remove an image from the collection at the specified index.

    - **index -** The zero-based index value of the image in the collection

- **remove_by_key(name: str) :** Remove the image with the specified key name from the collection.

    - **name: str -** The specified key name of the image (not case-sensitive).

- **set_key_name(index: int, name: str) :** Set the key name for an image in the collection.

    - **index -** The zero-based index value of the image in the collection.
    - **name -** The name to be set as the image's key name (not case-sensitive).

ImageList Usage Examples

This first example shows how the **ImageList** can be used to provide the "inactive" or "grayed" image for a Tkinter Button widget regardless of the computer's operating system. As explained earlier, when running on a **macOS** computer, a tkinter widget does not display a "grayed" image when that widget's **state** is set to **'disabled'**. In this example, the **ImageList.Grayed** collection is used provide the "grayed" image for the widget when running on a **macOS** platform.

```
import platform
import tkinter as tk
from imagelist import ImageList


class MacButton(tk.Button):
    """A macOS compatible image button widget."""

    def __init__(self, parent, image_list, image_id, command):
        """Create and initialize the macOS compatible image button widget."""
        self._grayed = self._image = image_list[image_id]
        if platform.system() == 'Darwin':  # Check for the macOS platform
            self._grayed = image_list.grayed[image_id]
        width, height = image_list.image_size
        super().__init__(parent, image=self._image, command=command)
        self.config(width=width * 1.25, height=height * 1.25)

    @property
    def enabled(self):
        """Get/Set the button enabled status."""
        return self['state'] != tk.DISABLED

    @enabled.setter
    def enabled(self, enabled):
        """Get/Set the button enabled status."""
        self['state'] = tk.NORMAL if enabled else tk.DISABLED
        self['image'] = self._image if enabled else self._grayed
```

```
class DemoForm(tk.Frame):
    """The MacButton demo window."""

    def __init__(self, master):
        """Construct the MacButton demo window."""
        super().__init__(master, bd=3, relief='ridge', padx=50, pady=20)
        self.grid()

        image_list = ImageList(image_size=(48, 48))
        image_list.add('image_folder/printer.png')  # Add an image of a printer
        self._button = MacButton(self, image_list, 0, self._on_print)
        self._button.enabled = False
        self._button.grid(pady=20)

        self._print_enable = tk.IntVar()
        checkbox = tk.Checkbutton(self, text='Enable Print Button')
        checkbox.config(variable=self._print_enable, command=self._on_check)
        checkbox.grid()

    def _on_check(self):
        """Event handler for the Checkbutton."""
        self._button.enabled = self._print_enable.get() != 0

    @staticmethod
    def _on_print():
        """Event handler for the MacButton."""
        print('*** Print ***')


if __name__ == '__main__':
    main_form = DemoForm(tk.Tk())
    main_form.mainloop()
```

This next example demonstrates how to use the **auto_load** option to create an instance of an **ImageList** class that automatically adds all the image files located in the specified **'image_folder'** ('Help.ico', 'none.ico', 'settings.ico', 'info.png', 'printer.png', 'Stop.png'). This script creates and displays a group of four image buttons. Each of these buttons is identified by the key name value of it's assigned image. Each button's event handler displays the name of that button when it is clicked.

```
import tkinter as tk
from imagelist import ImageList


class DemoForm(tk.Frame):
    """The image buttons demo window."""

    def __init__(self, master):
        """Construct the image buttons demo window."""
        super().__init__(master, bd=3, relief='ridge', padx=20, pady=20)
        self.grid()

        group = tk.LabelFrame(self, text='Image Buttons')
        image_list = ImageList('image_folder', (32, 32), True)
        image_width, image_height = image_list.image_size
        for i, name in enumerate(['printer', 'settings', 'help', 'stop']):

            def handler(index=image_list.index_of_key(name)):
                text = '???' if index < 0 else image_list.keys[index].title()
                label.configure(text=f"'{text}' Button")

            button = tk.Button(group, image=image_list[name], command=handler)
            button.configure(width=image_width + 8, height=image_height + 8)
            button.grid(column=i, row=0, padx=15, pady=(10, 15))
        group.grid(pady=(0, 20))

        label = tk.Label(self, bd=1, relief='solid', width=20, anchor='center')
        label.configure(text='Click any Button')
        label.grid()


if __name__ == '__main__':
    main_form = DemoForm(tk.Tk())
    main_form.mainloop()
```